Netflix Data Science Project

In [ ]:

In [35]:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import warnings

#Ignore all incoming warnings (if any):
warnings.filterwarnings("ignore")

# Load dataset
df = pd.read_csv("dataset/NetFlix.csv/NetFlix.csv")

# Copy original data
df_clean = df.copy()

# Standardize column names
df_clean.columns = (
    df_clean.columns
    .str.strip()
    .str.lower()
    .str.replace(" ", "_")
)

# Remove duplicates
df_clean.drop_duplicates(inplace=True)

# Replace fake null strings with actual NaN
df_clean.replace(['nan', 'None', ''], np.nan, inplace=True)

# Strip spaces from text columns
text_cols = ['show_id', 'type', 'title', 'director', 'cast', 'country', 'rating', 'genre
s', 'description']
for col in text_cols:
    if col in df_clean.columns:
        df_clean[col] = df_clean[col].astype(str).str.strip()
        df_clean[col] = df_clean[col].replace("nan", np.nan)

# Fill missing text values
for col in ['director', 'cast', 'country', 'rating', 'genres', 'description']:
    if col in df_clean.columns:
        df_clean[col] = df_clean[col].fillna('Unknown')

# Convert date_added to datetime
df_clean['date_added'] = pd.to_datetime(df_clean['date_added'], errors='coerce', dayfirs
t=True)

# Fill missing date_added with median date
median_date = df_clean['date_added'].dropna().median()
df_clean['date_added'] = df_clean['date_added'].fillna(median_date)

# Ensure release_year is numeric
df_clean['release_year'] = pd.to_numeric(df_clean['release_year'], errors='coerce')
df_clean['release_year'] = df_clean['release_year'].fillna(df_clean['release_year'].media
n())

# Clean duration column: extract numeric value from strings like "90 min", "2 Seasons"
df_clean['duration'] = (
    df_clean['duration']
    .astype(str)
    .str.extract(r'(\d+)')[0]
```

```
)
df_clean['duration'] = pd.to_numeric(df_clean['duration'], errors='coerce')
df_clean['duration'] = df_clean['duration'].fillna(df_clean['duration'].median())

# Clean comma-separated text columns
def clean_comma_separated_text(x):
    if pd.isna(x) or str(x).strip().lower() == 'unknown':
        return 'Unknown'
    parts = [item.strip() for item in str(x).split(',')]
    return ', '.join(parts)

for col in ['cast', 'country', 'genres']:
    df_clean[col] = df_clean[col].apply(clean_comma_separated_text)

# Feature engineering
df_clean['added_year'] = df_clean['date_added'].dt.year
df_clean['added_month'] = df_clean['date_added'].dt.month
df_clean['added_day'] = df_clean['date_added'].dt.day

df_clean['cast_count'] = df_clean['cast'].apply(lambda x: 0 if x == 'Unknown' else len([
i for i in str(x).split(',') if i.strip()]))
df_clean['country_count'] = df_clean['country'].apply(lambda x: 0 if x == 'Unknown' else
len([i for i in str(x).split(',') if i.strip()]))
df_clean['genre_count'] = df_clean['genres'].apply(lambda x: 0 if x == 'Unknown' else le
n([i for i in str(x).split(',') if i.strip()]))
df_clean['description_length'] = df_clean['description'].apply(lambda x: len(str(x).strip
()))

# Final checks
print("Final shape:", df_clean.shape)
print("\nMissing values after cleaning:\n")
print(df_clean.isnull().sum())

print("\nData types:\n")
print(df_clean.dtypes)
```

```
Final shape: (7787, 19)

Missing values after cleaning:

show_id                 0
type                    0
title                   0
director                0
cast                    0
country                 0
date_added              0
release_year            0
rating                  0
duration                0
genres                  0
description             0
added_year              0
added_month             0
added_day               0
cast_count              0
country_count           0
genre_count             0
description_length      0
dtype: int64

Data types:

show_id                       object
type                          object
title                         object
director                      object
cast                          object
country                       object
date_added             datetime64[ns]
release_year                    int64
rating                        object
```

```
duration                        int64
genres                         object
description                    object
added_year                      int32
added_month                     int32
added_day                       int32
cast_count                      int64
country_count                   int64
genre_count                     int64
description_length              int64
dtype: object
```

In [36]:

```
df_clean.shape
```

Out[36]:

```
(7787, 19)
```

In [37]:

```
df_clean.describe()
```

Out[37]:

	date_added	release_year	duration	added_year	added_month	added_day	cast_count	country_count	g
count	7787	7787.000000	7787.000000	7787.000000	7787.000000	7787.000000	7787.000000	7787.000000	7
mean	2019-01-02 21:19:40.274817024	2013.932580	69.122769	2018.494029	6.778991	12.372159	7.185694	1.163863	
min	2008-01-01 00:00:00	1925.000000	1.000000	2008.000000	1.000000	1.000000	0.000000	0.000000	
25%	2018-02-01 00:00:00	2013.000000	2.000000	2018.000000	4.000000	1.000000	4.000000	1.000000	
50%	2019-03-08 00:00:00	2017.000000	88.000000	2019.000000	7.000000	12.000000	8.000000	1.000000	
75%	2020-01-17 12:00:00	2018.000000	106.000000	2020.000000	10.000000	20.000000	10.000000	1.000000	
max	2021-01-16 00:00:00	2021.000000	312.000000	2021.000000	12.000000	31.000000	50.000000	12.000000	
std	NaN	8.757395	50.950743	1.387371	3.591858	9.951626	4.859990	0.732262	

In [38]:

```
df_clean.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7787 entries, 0 to 7786
Data columns (total 19 columns):
 #   Column              Non-Null Count  Dtype
---  ------              --------------  -----
 0   show_id             7787 non-null   object
 1   type                7787 non-null   object
 2   title               7787 non-null   object
 3   director            7787 non-null   object
 4   cast                7787 non-null   object
 5   country             7787 non-null   object
 6   date_added          7787 non-null   datetime64[ns]
 7   release_year        7787 non-null   int64
 8   rating              7787 non-null   object
 9   duration            7787 non-null   int64
 10  genres              7787 non-null   object
 11  description         7787 non-null   object
 12  added_year          7787 non-null   int32
 13  added_month         7787 non-null   int32
 14  added_day           7787 non-null   int32
```

```
 15   cast_count          7787 non-null   int64
 16   country_count       7787 non-null   int64
 17   genre_count         7787 non-null   int64
 18   description_length  7787 non-null   int64
dtypes: datetime64[ns](1), int32(3), int64(6), object(9)
memory usage: 1.0+ MB
```

In [ ]:

In [ ]:

Q1] Netflix wants to segment its content catalog into groups based on content characteristics such as release year and duration.

In [39]:

```
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

# Numeric features for clustering
feature_cols = [
    "release_year",
    "duration",
    "genre_count",
    "cast_count",
    "country_count",
    "description_length"
]

X = df_clean[feature_cols].copy()

# Scale features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Train KMeans
kmeans = KMeans(n_clusters=4, random_state=42, n_init=10)
clusters = kmeans.fit_predict(X_scaled)

# Add cluster ids
df_clean["cluster"] = clusters

# Cluster summary
cluster_summary = df_clean.groupby("cluster")[feature_cols].mean().round(2)

print("Cluster Summary:\n")
print(cluster_summary)

# Build interpretable labels automatically
overall_medians = df_clean[feature_cols].median()

def make_cluster_label(row, medians):
    parts = []

    if row["release_year"] >= medians["release_year"]:
        parts.append("Recent")
    elif row["release_year"] < medians["release_year"] - 10:
        parts.append("Very Old")
    else:
        parts.append("Mid-Age")

    if row["duration"] >= medians["duration"]:
        parts.append("Long")
    else:
        parts.append("Short")
```

```
    richness_score = 0
    if row["genre_count"] >= medians["genre_count"]:
        richness_score += 1
    if row["cast_count"] >= medians["cast_count"]:
        richness_score += 1
    if row["description_length"] >= medians["description_length"]:
        richness_score += 1

    if richness_score >= 2:
        parts.append("Content-Rich")
    else:
        parts.append("Simple")

    if row["country_count"] >= medians["country_count"]:
        parts.append("Global")

    return ", ".join(parts)

cluster_name_map = {
    cluster_id: make_cluster_label(cluster_summary.loc[cluster_id], overall_medians)
    for cluster_id in cluster_summary.index
}

print("\nInterpretable Cluster Labels:\n")
for cluster_id, label in cluster_name_map.items():
    print(f"Cluster {cluster_id}: {label}")

df_clean["cluster_name"] = df_clean["cluster"].map(cluster_name_map)

# PCA for 2D visualization
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

# Transform centroids into PCA space
centers_pca = pca.transform(kmeans.cluster_centers_)

# Plot clusters
plt.figure(figsize=(15, 10))
colors = ["orange", "purple", "blue", "green"]

for i, cluster_id in enumerate(cluster_summary.index):
    plt.scatter(
        X_pca[clusters == cluster_id, 0],
        X_pca[clusters == cluster_id, 1],
        s=40,
        color=colors[i % len(colors)],
        label=cluster_name_map[cluster_id]
    )

plt.scatter(
    centers_pca[:, 0],
    centers_pca[:, 1],
    s=200,
    c="red",
    alpha=0.7,
    label="Centroids"
)

plt.title("Contents PCA Projection")
plt.xlabel("PCA Component 1")
plt.ylabel("PCA Component 2")
plt.legend()
plt.show()

print("\nCluster Counts:\n")
print(df_clean["cluster_name"].value_counts())

cluster_profile = cluster_summary.copy()
cluster_profile["cluster_label"] = cluster_profile.index.map(cluster_name_map)

print("\nCluster Profile with Labels:\n")
```

```
print(cluster_profile)
```

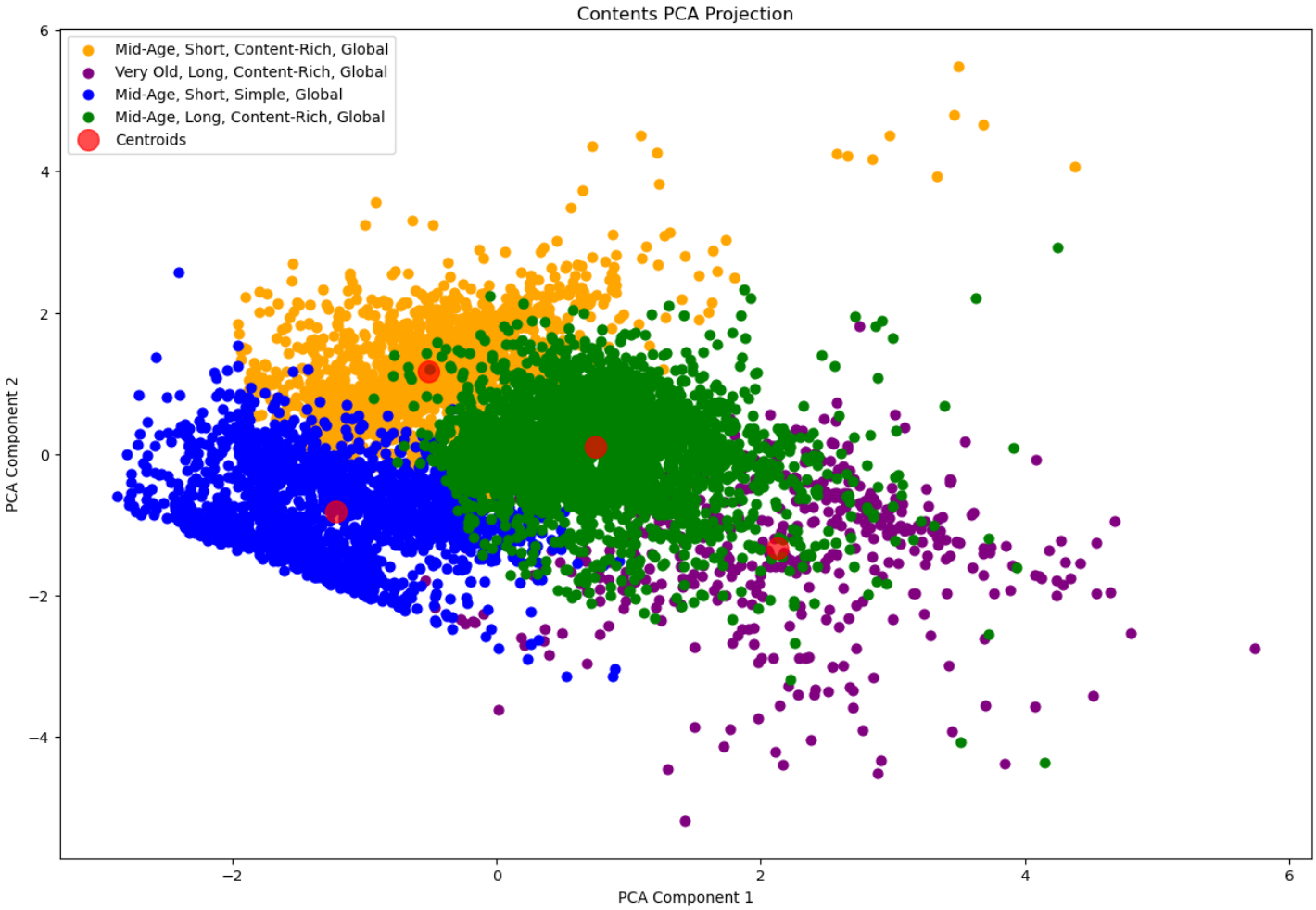
```
Cluster Summary:

         release_year  duration  genre_count  cast_count  country_count  \
cluster
0             2016.56      3.21         2.68        8.82           1.00
1             1986.35    108.02         2.31        8.65           1.17
2             2016.48     52.25         1.27        3.34           1.01
3             2014.69    108.36         2.51        8.58           1.34

         description_length
cluster
0                    142.35
1                    150.21
2                    143.21
3                    142.46

Interpretable Cluster Labels:

Cluster 0: Mid-Age, Short, Content-Rich, Global
Cluster 1: Very Old, Long, Content-Rich, Global
Cluster 2: Mid-Age, Short, Simple, Global
Cluster 3: Mid-Age, Long, Content-Rich, Global
```




```
Cluster Counts:

cluster_name
Mid-Age, Long, Content-Rich, Global     3415
Mid-Age, Short, Simple, Global          2161
Mid-Age, Short, Content-Rich, Global    1751
Very Old, Long, Content-Rich, Global     460
Name: count, dtype: int64

Cluster Profile with Labels:

         release_year  duration  genre_count  cast_count  country_count  \
cluster
0             2016.56      3.21         2.68        8.82           1.00
1             1986.35    108.02         2.31        8.65           1.17
```

```
1              1986.55     106.02          2.51          8.05            1.17
2              2016.48      52.25          1.27          3.34            1.01
3              2014.69     108.36          2.51          8.58            1.34

         description_length                          cluster_label
cluster
0                    142.35  Mid-Age, Short, Content-Rich, Global
1                    150.21  Very Old, Long, Content-Rich, Global
2                    143.21        Mid-Age, Short, Simple, Global
3                    142.46   Mid-Age, Long, Content-Rich, Global
```

In [40]:

```
pca_df = pd.DataFrame(
    pca.components_,
    columns=feature_cols,
    index=["PC1", "PC2"]
)

pca_df
```

Out[40]:

	release_year	duration	genre_count	cast_count	country_count	description_length
PC1	-0.457741	0.543020	0.397026	0.470831	0.326197	0.099431
PC2	0.337583	-0.348746	0.612369	0.458132	-0.247492	-0.343921

CONCLUSION] The data suggests that older content is often longer and more complex, while newer content tends to be relatively shorter and simpler.

In [ ]:

```

```

In [ ]:

```

```

Q2] Can we classify whether a Netflix title is 'Content-Rich' or 'Simple' based on features like duration, cast size, genres, and description length?

In [41]:

```
df_clean["content_type"] = df_clean["genre_count"].apply(
    lambda x: "Content-Rich" if x >= df_clean["genre_count"].median() else "Simple"
)

from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score, classification_report

X = df_clean[[
    "duration",
    "cast_count",
    "country_count",
    "description_length"
]].copy()

y = df_clean["content_type"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

model = DecisionTreeClassifier(max_depth=4, random_state=42)
model.fit(X_train, y_train)
```

```
y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n")
print(classification_report(y_test, y_pred))

plt.figure(figsize=(18, 8))
plot_tree(
    model,
    feature_names=X.columns,
    class_names=model.classes_,
    filled=True
)
plt.show()
```

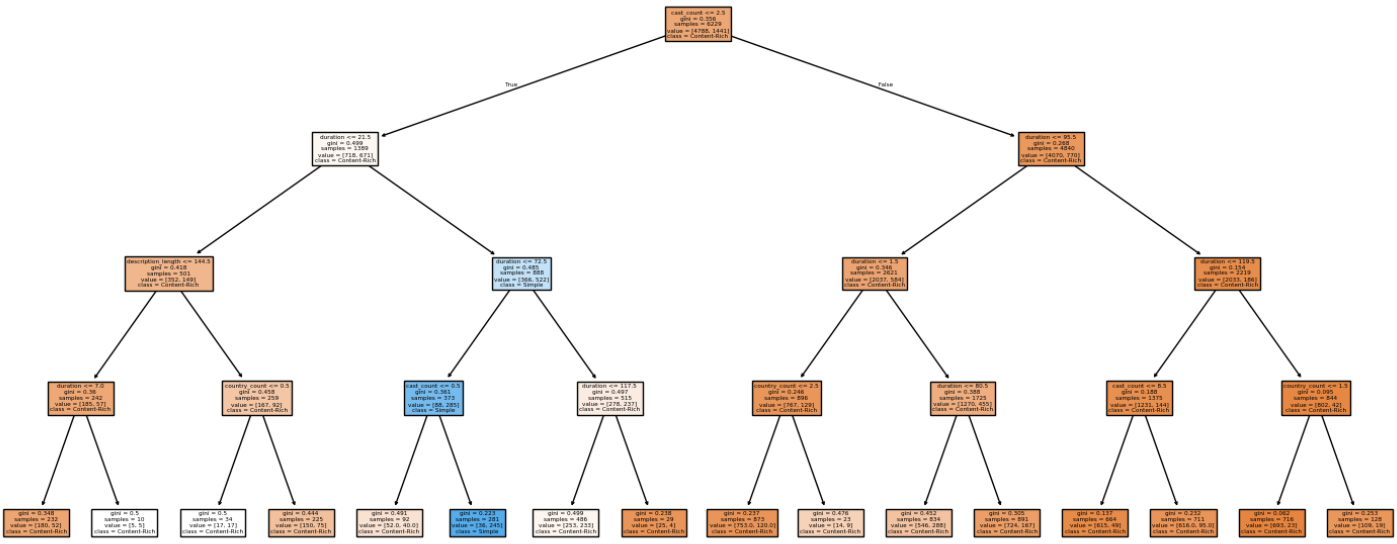
```
Accuracy: 0.8042362002567394

Classification Report:

               precision    recall  f1-score   support

Content-Rich       0.80      0.99      0.89      1198
      Simple       0.91      0.17      0.29       360

    accuracy                           0.80      1558
   macro avg       0.85      0.58      0.59      1558
weighted avg       0.83      0.80      0.75      1558
```



In [ ]:

Q3] Netflix claims that the average duration of content is 100 minutes. Is it true?

In [42]:

```
from statsmodels.stats.weightstats import ztest

sample = df_clean[df_clean['release_year'] >= 2020]['duration'].dropna()

z_stat, p_value = ztest(sample, value=100)

# Left-tailed test: H1 = average duration < 100
if z_stat < 0:
    p_value = p_value / 2
else:
    p_value = 1 - (p_value / 2)
```

```
print("Z-Statistic:", z_stat)
print("P-Value:", p_value)

alpha = 0.05
if p_value < alpha:
    print("\nReject H0")
    print("Modern content is significantly shorter than 100 minutes.")
else:
    print("\nFail to Reject H0")
    print("There is not enough evidence that modern content is shorter than 100 minutes."
)
```

```
Z-Statistic: -34.96744134136605
P-Value: 3.5171670321817373e-268

Reject H0
Modern content is significantly shorter than 100 minutes.
```

In [ ]:

```

```

Q4] Is there a statistically significant difference in the average duration of Netflix Movies vs TV Shows?

In [43]:

```
from scipy.stats import ttest_ind

movies = df_clean[df_clean["type"] == "Movie"]['duration'].dropna()
tv_shows = df_clean[df_clean["type"] == "TV Show"]['duration'].dropna()

t_stat, p_value = ttest_ind(movies, tv_shows, equal_var=False)

print("T-Statistic:", t_stat)
print("P-Value:", p_value)

alpha = 0.05

if p_value < alpha:
    print("\nReject H0")
    print("There is a statistically significant difference in average duration.")
else:
    print("\nFail to Reject H0")
    print("No significant difference in average duration.")
```

```
T-Statistic: 249.79900942857398
P-Value: 0.0

Reject H0
There is a statistically significant difference in average duration.
```

In [ ]:

```

```

Q5] Is there a statistically significant difference in the average duration of Netflix content across different genres?

In [44]:

```
from scipy.stats import f_oneway

def extract_main_genre(x):
    if pd.isna(x):
        return 'Other'
    x = str(x).lower()
```

```
    if 'drama' in x:
        return 'Drama'
    elif 'comedy' in x:
        return 'Comedy'
    elif 'documentary' in x:
        return 'Documentary'
    else:
        return 'Other'

df_clean['main_genre'] = df_clean['genres'].apply(extract_main_genre)

anova_df = df_clean[df_clean['main_genre'].isin(['Drama', 'Comedy', 'Documentary'])].copy
()

drama = anova_df[anova_df['main_genre'] == 'Drama']['duration'].dropna()
comedy = anova_df[anova_df['main_genre'] == 'Comedy']['duration'].dropna()
documentary = anova_df[anova_df['main_genre'] == 'Documentary']['duration'].dropna()

f_stat, p_value = f_oneway(drama, comedy, documentary)

print("F-Statistic:", f_stat)
print("P-Value:", p_value)

alpha = 0.05

if p_value < alpha:
    print("\nReject Null Hypothesis (H0)")
    print("There is a significant difference in average duration across genres.")
else:
    print("\nFail to Reject Null Hypothesis (H0)")
    print("No significant difference in average duration across genres.")
```

```
F-Statistic: nan
P-Value: nan

Fail to Reject Null Hypothesis (H0)
No significant difference in average duration across genres.
```

In [ ]:

```

```

Q6] Is there a statistically significant association between content type (Movie vs TV Show) and content rating category (for example Kids, Teens, Adults) on Netflix?

In [45]:

```
from scipy.stats import chi2_contingency

def rating_category(rating):
    kids = ['G', 'TV-G', 'TV-Y', 'TV-Y7']
    teens = ['PG', 'TV-PG', 'PG-13', 'TV-14']
    adults = ['R', 'NC-17', 'TV-MA']

    if rating in kids:
        return 'Kids'
    elif rating in teens:
        return 'Teens'
    elif rating in adults:
        return 'Adults'
    else:
        return 'Other'

df_clean['rating_group'] = df_clean['rating'].apply(rating_category)

cont_table = pd.crosstab(df_clean['type'], df_clean['rating_group'])

print("Contingency Table:\n")
print(cont_table)
```

```
chi2_stat, p_value, dof, expected = chi2_contingency(cont_table)

print("\nChi-Square Statistic:", chi2_stat)
print("P-Value:", p_value)
print("Degrees of Freedom:", dof)

alpha = 0.05

if p_value < alpha:
    print("\nReject Null Hypothesis (H0)")
    print("There is a significant association between content type and rating category.")
else:
    print("\nFail to Reject Null Hypothesis (H0)")
    print("No significant association between content type and rating category.")
```

```
Contingency Table:

rating_group  Adults  Kids  Other  Teens
type
Movie           2511   362     94   2410
TV Show         1020   422      8    960

Chi-Square Statistic: 234.07533384399608
P-Value: 1.8182290180291904e-50
Degrees of Freedom: 3

Reject Null Hypothesis (H0)
There is a significant association between content type and rating category.
```

In [ ]:

```

```

Q7] Netflix historically claims that the average cast size is 10 actors per title. Using recently added content (added_year >= 2020), test whether the average cast size is significantly lower than 10.

In [46]:

```
from statsmodels.stats.weightstats import ztest

recent_content = df_clean[df_clean["added_year"] > 2019]["cast_count"].dropna()

z_stat, p_value = ztest(recent_content, value=10)

# Left-tailed test: H1 = average cast count < 10
p_value = p_value / 2

print("Z-Statistic:", z_stat)
print("P-Value:", p_value)

alpha = 0.05

if p_value < alpha:
    print("\nReject H0")
    print("Average cast is significantly lower than 10.")
else:
    print("\nFail to Reject H0")
    print("Average cast is not significantly lower than 10.")
```

```
Z-Statistic: -19.766052015675466
P-Value: 2.918439740348507e-87

Reject H0
Average cast is significantly lower than 10.
```

In [ ]:

```

```

In [ ]:

Conclusion:

This project analyzed Netflix content data using a combination of exploratory data analysis, machine learning models, clustering, classification, and statistical hypothesis testing to derive meaningful insights about content patterns and platform strategy.

The data was first cleaned and transformed by handling missing values, standardizing text fields, converting dates, and engineering useful features such as cast count, country count, genre count, and description length. The duration column was also cleaned into numeric form for consistent analysis.

Using K-Means clustering, Netflix content was segmented into meaningful groups based on release year, duration, genre diversity, cast size, country spread, and description length. This revealed that some clusters were more recent and simple, while others were older, longer, and more content-rich.

A classification model was then built to predict whether a title is “Content-Rich” or “Simple” using structural metadata. This showed that traditional machine learning can help identify content categories from existing dataset features.

Statistical hypothesis testing was also applied to validate several assumptions:

- **A Z-test was used to examine whether recent Netflix content is shorter than the claimed average duration.**
- **A T-test was used to compare the average duration of Movies and TV Shows.**
- **ANOVA was used to test whether average duration differs across major genres.**
- **A Chi-square test was used to check whether content type and rating category are associated.**
- **Another Z-test was used to test whether recent Netflix titles have a smaller cast size than the historical claim.**

Overall, the project shows that traditional machine learning and hypothesis testing can be combined effectively to generate business-relevant insights from entertainment data. The findings suggest that Netflix content can be meaningfully segmented, compared, and tested using structured metadata, helping support data-driven content strategy and catalog understanding.

In [ ]:

In [ ]: